Compiler Documentation

submitted by: Guy Vinitzky 208848499, Roy Bar-On 209161439

1. Compiler Overview

The compiler translates programs written in **C--**, a subset of the C programming language, into **Riski** assembly language. The **C--** language includes the following features:

- **Primitive types**: int, float, and void.
- **Control structures**: if, if-else, while, do-while.
- **Function calls**: Including support for variadic functions (...) and the va_arg construct.
- **Static scoping**: Variables are scoped to the block in which they are declared.

Limitations:

- No support for pointers, structs, no global variables. all variables are local to their respective blocks or functions.

Variadic Functions:

- The compiler enforces that the first argument to a variadic function call specifies the number of variadic arguments passed. This is checked at runtime using the va_arg construct.
- Variadic arguments can be of any type (int or float), but the number of arguments must match the value of the first parameter.

1.2 Design Philosophy

The compiler is implemented as a **multi-pass, bottom-up LR parser** using **Bison** and **Flex**. The design is modular, with clear separation between lexical analysis, syntax analysis, semantic analysis, and code generation. The key design principles include:

1. **Grammar-Driven Translation**: The compiler uses Bison to implement grammar rules for parsing and semantic actions for code generation.
2. **Backpatching**: A technique used to handle forward jumps in control flow and function calls efficiently.
3. **Static Scoping**: Enforced using symbol tables to manage variable declarations and lookups.
4. **Intermediate Representation (IR)**: Directly generates Riski assembly code, simplifying the compilation process.

Multi-Pass Compilation:

- **Pass 1 (Lexical Analysis)**: Transforms the source code into a stream of tokens using Flex.
- **Pass 2 (Syntax Analysis)**: Parses the token stream into an Abstract Syntax Tree (AST) using Bison, based on the C-- grammar.
- **Pass 3 (Semantic Analysis)**: Performs type checking, scope resolution, and other semantic checks using the symbol table and function table.
- **Pass 4 (Intermediate Code Generation)**: Generates Riski assembly code as the intermediate representation (IR).

Error Handling:

- **Lexical errors**: Detected during tokenization and reported with the offending lexeme and line number.
- **Syntax errors**: Detected during parsing and reported with the problematic token and line number.
- **Semantic errors**: Detected during semantic analysis, such as type mismatches or undefined variables, and reported with detailed descriptions.
- **Runtime errors**: Detected during execution of the generated code (e.g., invalid va_arg indices) and reported with a "Runtime error!" message followed by a HALT instruction.

2. Data Structures for Compiler State

2.1 Symbol Table

The **symbol table** is implemented using a std::map<std::string, Symbol> from the C++ Standard Template Library (STL). Each entry in the symbol table maps a variable name to a Symbol object, which stores information about the variable across different scopes.

Structure of a Symbol:

- type: A std::map<int, Type> mapping scope levels to the variable's type (int, float, or void).
- offset: A std::map<int, int> mapping scope levels to the variable's memory offset relative to the frame pointer.
- depth: The maximum scope depth at which the variable is defined.

Scope Management:

- **Entering a new scope**: Increment currentScopeOffset.
- **Exiting a scope**: Remove all symbols defined at the current scope level and update their depth to the next outermost scope.

Example:

```
int main() {
  x: int;      // depth = 1, offset = -8
  {
    x: float;  // depth = 2, offset = -12
  }
}
```

Symbol Table Evolution:

1. After x: int :

   Symbol Table:
   ```
   {
    "x": { type: {1: int_}, offset: {1: -8}, depth: 1 }
   }
   ```

2. After x: float :

   Symbol Table:
   ```
   {
    "x": { type: {1: int_, 2: float_}, offset: {1: -8, 2: -12}, depth: 2 }
   ```

3. After exiting the inner block:

   Symbol Table:
   ```
   {
    "x": { type: {1: int_}, offset: {1: -8}, depth: 1 }
   }
   ```

2.2 Function Table

The **function table** is implemented as a std::map<std::string, Function>. Each entry maps a function name to a Function object, which stores information about the function's declaration and definition.

Structure of a Function:

- isDefined: A flag indicating whether the function is defined.
- startLineImplementation: The line number where the function is implemented (-1 Indicate that the function is declared but not defined yet)
- paramTypes: A vector of parameter types.
- callingLines: A list of line numbers where the function is called.
- paramIds: A vector of parameter names.
- returnType: The return type of the function (int, float, or void).

Variadic Functions:

- indexNumVAParams: The index of the parameter indicating the number of variadic arguments.
- startIndexVAParams: The starting index of the variadic arguments.

For both of them -1 Indicate that the function is not variadic function.

Example:

```
float sum(n: int, ...) {
  return va_arg(float, n-1) + va_arg(float, n-2);
}
```

Function Table Entry:

```
{
  "sum": {
    returnType: float_,
    paramTypes: [int_],
    paramIds: ["n"],
    indexNumVAParams: 1,
    startIndexVAParams: 2,
    isDefined: true
  }
}
```

2.3 Buffer

The **buffer** is a custom class (Buffer) that stores the generated Riski assembly code. It provides methods for emitting instructions, backpatching, and inserting headers.

Key Methods:

- emit(const std::string&): Appends a new instruction to the buffer.
- frontEmit(const std::string&): Inserts a header at the beginning of the buffer.
- backpatch(const std::vector<int>&, int): Resolves placeholders for jump targets.

Example:

Line 5: buffer->emit("BREQZ I1 ");
buffer->backpatch(5, 10); // Resolves the placeholder at line 5 to line 10.

3. Backpatching and Code Generation

3.1 Backpatching Technique

Backpatching is a technique used in the compiler to handle forward references in control flow and function calls. It allows the compiler to generate code for control structures and function calls without knowing the exact jump targets at the time of code generation. These targets are resolved later during the backpatching phase.

Control Structures

1. **if Statement**:
    a. The **trueList** of the boolean expression is backpatched to the start of the if body.
    b. The **falseList** is merged with the **nextList** of the if body and passed to the next reduce for further processing.
2. **if-else Statement**:
    a. The **trueList** of the boolean expression is backpatched to the start of the if body.
    b. The **falseList** is backpatched to the start of the else body.
    c. The **nextList** of both branches is merged and passed to the next reduce.
3. **while Loop**:
    a. The **trueList** of the boolean expression is backpatched to the start of the loop body.
    b. The **falseList** is passed as the **nextList** to the next reduce.
    c. The **nextList** of the loop body is backpatched to the start of the boolean expression to create the loop.
4. **Exiting a Block**:
    a. When reducing a block (BLK), the **nextList** of the block is backpatched to the line after the block in the buffer.

Function Calls

1. **Unresolved Calls**:
    a. When a function is called but not yet defined, a placeholder jump (JLINK _) is emitted, and the line number is stored in the function's callingLines list in the function table.
2. **Backpatching Function Calls**:
    a. At the end of the program, all unresolved function calls are backpatched using the callingLines list and the startLineImplementation of each function in the function table.

Example of Backpatching

```
if (x < y) then
        x = x + 1;
```

IR Before Backpatching:

```
SLETI I5 I3 I4
BREQZ I5 _
UJUMP _
```

IR After Backpatching:

```
SLETI I5 I3 I4
BREQZ I5 12
UJUMP 8
```

4. Activation Records

4.1 Structure

The activation record is stored on the stack and is used to manage the state of a function call. It includes the following components:

- **Return Address**: Stored in register I0. This is the address to which the program will return after the function call completes.
- **Frame Pointer**: Stored in register I1. Points to the base of the current activation record and is used to access function parameters and local variables.
- **Stack Pointer**: Stored in register I2. Points to the top of the stack and is updated as memory is allocated or deallocated during the function's execution.
- **Function Parameters**: Stored at offsets from I1, starting at I1 - 8 for the first parameter and growing downward.
- **Local Variables**: Stored at offsets from I2, growing upward as memory is allocated.

Variadic Functions and Register I3

For variadic functions, the first parameter (in function call) is always an integer (n: int) that specifies the number of variadic arguments. This parameter is passed as the first argument in the function call and is always the last parameter in the function definition. The compiler uses register I3 to store the offset of n relative to the frame pointer (I1). This allows the function to dynamically access the variadic arguments during execution.

- **Accessing Variadic Arguments**:
  The variadic arguments are stored sequentially on the stack, starting at the offset immediately after the fixed parameters. The compiler calculates the offset of each variadic argument using the value of n stored at I3. For example:
  - The first variadic argument is at I1 -8 - 4 * (n - 1).
  - The second variadic argument is at I1 -8 - 4 * (n - 2), and so on.
- **Runtime Checks**:
  The compiler ensures that the index used to access a variadic argument is within the valid range [0, n-1]. If the index is out of bounds, a runtime error is raised, and the program halts. This structure ensures efficient management of variadic arguments while maintaining compatibility with the standard activation record layout.

5. Register Allocation

The activation record is stored on the stack and includes:

- **Return address**: Stored in I0.
- **Frame pointer**: Stored in I1.
- **Stack pointer**: Stored in I2.
- **Function parameters**: Stored at offsets from I1.
- **Local variables**: Stored at offsets from I2.

Temporary Registers

- Temporary registers are allocated sequentially starting from I3. When a temporary value is no longer needed, its register is made available for reuse.

6. Compiler Modules

6.1 Lexer

- **Purpose**: Tokenizes the input source code.
- **Implementation**: Written in Flex, using regular expressions to identify tokens.

6.2 Parser

- **Purpose**: Parses the token stream and builds the syntax tree.
- **Implementation**: Written in Bison, using an LR grammar.

6.3 Semantic Analyzer

- **Purpose**: Enforces semantic rules (e.g., type checking, scope resolution).
- **Implementation**: Uses symbol tables and function tables.

6.4 Code Generator

- **Purpose**: Generates Riski assembly code.
- **Implementation**: Emits instructions to the buffer during parsing.

6.5 Helper files

- **Purpose**: part3_helper.hpp and part3_helper.cpp provide the core data structures, utility functions, and global variables required for the implementation of the compiler.
- **Implementation**:
    - part3_helper.hpp: Defines the Symbol and Function classes for symbol and function tables, the Buffer class for code generation, and utility functions like merge (for backpatching) and intToString. It also declares global variables (symbolTable, functionTable, buffer) and the yystype struct for passing attributes during parsing.
    - part3_helper.cpp: Implements Buffer methods (emit, frontEmit, backpatch, nextQuad), initializes the global Buffer object, and handles backpatching and code generation logic.

7. External Data Structures

The compiler uses the C++ STL for std::map and std::vector. These were chosen for their efficiency and ease of use.